

Pizza-To-Hell

Development Log · PSX-inspired melee combat game · Unity 6 (HDRP)

Programming, editor tooling & VFX/UI — development work only

NEW

feature

FIX

bug fix

REFACTOR

restructure

REVERTED

rolled back

Phase 1 · Architecture & Foundation

Mar 29

NEW

Combat system architecture design

- Designed a modular, manager-driven melee combat system before writing code: player light/heavy attacks, block, dodge, parry; three enemy types and three weapon types.
- Established a single `AgentState` enum (with `LightAttacking/HeavyAttacking` as states), `WeaponBase` as a composition reference rather than inheritance, and a singleton `CombatManager` owning all hit resolution.
- Decided hitbox activation would be data-driven; parry framed as a defender-side timed state. Buff/debuff system outlined conceptually via a stat-modifier stack.

Mar 29

NEW

FIX

Interface & core script implementation

- Wrote the four interfaces (`IDamageable`, `IAttacker`, `IBlockable`, `IStateMachine`), abstract `Agent/WeaponBase`, concrete `Sword`, the handler trio (`HitboxManager`, `HurtboxHandler`, `AttackTypeHandler`), `DamageInfo` struct and `WeaponDataSO`.
- Built the full player stack: `PlayerAgent`, `PlayerCombatActions`, `PlayerController`. Region blocks + XML docs set as a standing convention.
- Cleaned up file issues (stray markdown fences, empty files, a duplicate `WeaponData/WeaponDataSO` class) and fixed a redundant ternary bug in `CombatManager.BuildDamageInfo()`.

Mar 30

NEW

FIX

HDRP project setup & click-to-damage pipeline

- Stood up a fresh HDRP project; resolved an Input System package conflict (Active Input Handling → Both) and an abstract-class blocker via an `EnemyStub`.
- Added `PlayerDebugger` with toggleable inspector flags and a manual T-key hitbox test coroutine.
- Verified the full pipeline end to end: input → `PlayerCombatActions` → state change → `Sword.PerformAttack()` → hitbox → hurtbox → `CombatManager.ResolveHit()` → enemy HP down. Stamina system later stripped out.

Phase 2 · Animation & Combo System

Apr 1

NEW FIX

Light attack animation pipeline

- Built the data-driven attack pipeline: `AttackData`, `ComboSequence`, `WeaponLoadout` SOs plus `PlayerAnimationController`, `AttackAnimationDriver`, `ComboHandler`.
- Switched from trigger/index Animator transitions to direct `animator.Play(stateName)`, eliminating magic-number indices; made the driver the sole authority on the `Idle` transition.
- Fixed premature idle transitions and double-firing hitboxes with a `hitboxWindowDone` flag and lifecycle ownership inside the coroutine.

Apr 2

NEW

Weapon Builder editor tool — design

- Planned a unified `EditorWindow` weapon-authoring pipeline (“one-stop shop”): custom PropertyDrawers, hitbox gizmos, combo-tree visualizer.
- Confirmed the timer-based hitbox approach in `AttackData` is architecturally correct over baked Animation Events for a data-driven design. Defined extension points for VFX and parry.

Apr 3

REFACTOR FIX NEW

Combo upgrade — sequence-walking architecture

- Replaced the `nextAttacks[]` array chaining with `ComboStep` objects walked inside `ComboSequence` SOs.
- Traced an attack-cancel bug to a same-frame race between coroutine finish and input; fixed with a one-frame `yield return null` buffer before nulling the active coroutine.
- Replaced error-prone manual `clipDuration` with a direct `AnimationClip` reference (`clip.length`). Created `AttackDataEditor.cs` with a read-only clip length and a visual hitbox/combo-window timeline; added `WeaponBase` scene gizmos.

Apr 9

NEW FIX

Input buffering + dodge mechanics

- Implemented combo input buffering across `ComboHandler`, `AttackAnimationDriver`, `PlayerCombatActions` — animations always finish before a buffered input commits via `CommitBufferedAttack()`.
- Built the dodge system: `DodgeData` SO (curve-driven distance, i-frame window, two-step timing) and `DodgeHandler` (direction capture on press, velocity zeroing, i-frames, scene gizmos).
- Added an i-frame check at the top of `CombatManager.ResolveHit()`; fixed an attack stuck on its last frame via Animator exit-time transitions.

Phase 3 · Editor Tooling

Apr 19

REFACTOR

AttackDataEditor restructure

- Split the monolithic editor script into a clean hierarchy under `Assets/Editor/AttackDataEditor/`: `AttackEditorStyles`, `TimelineDrawer`, `PhaseSectionDrawer`, `AttackPreviewController`, plus a partial-class main entry.
- Resolved a `CS0103` cross-reference compile error by keeping all partials and referenced classes in one assembly scope (no `.asmdef`) and blanking the old root file.

Apr 20

NEW

Parry fine-tune tab in editor tool

- Ported the attack system's "Fine-tune" tab to the parry section in both `AttackTimingToolWindow` and `GizmoAndParryEditors`: center-zero sliders for startup/impact/recovery persisted per-asset via `EditorPrefs` GUID keys, plus a reset button.
- Resolved a transport-button glyph rendering in color due to a hidden emoji variant selector (`U+FE0F`).

Phase 4 · Parry System & Enemy Combat

Apr 20

REVERTED

Enemy attack animation via `ScriptableObject`

- Attempted `EnemyAttackDataSO` mirroring the player pattern; FBX-sourced clip had baked Y-axis root motion causing enemies to fall through the floor.
- After several import/bake fixes failed, fully reverted to the original serialized fields to keep the AI state machine and token timing intact.

Apr 21–22

REFACTOR

FIX

NEW

Parry feedback, refactor & enemy parried state

- Audited all `CombatEvents` subscribers; added centralized parry feedback (particles + audio) in `CombatManager`, keeping `PoolManager` as dumb infrastructure. Created the `ParrySparkBurst` prefab.
- Merged `ParryWindowHandler` + `ParryBoxHandler` into a single `ParryHandler` that fires one event; `EnemyAI` and `WeaponVFX` subscribe independently.
- Built a dedicated `Parried` state directly in `EnemyAI` (separate from `Staggered`) with a recovery coroutine; gutted dead `ParriedState/StaggerCause` files.
- Diagnosed the core gameplay bug: `parryBoxStartTime` (~1s) meant the parry collider activated far too late; fixed particle one-shot re-trigger (remove inline `Stop()`, `SetActive(true)` before `Play()`). Imported and rig-matched a Mixamo parry animation.

Phase 5 · Streak HUD & VFX

May 21–22

NEW

FIX

StreakUI combo HUD

- Designed and built the combo streak HUD: each committed attack spawns an icon that flies from a fixed origin into a row slot (`StreakUI`, `StreakRowUI`, `StreakIconUI`).
- Added new `CombatEvents` (`OnAttackStartedWithIcon`, `OnComboCompleted`, `OnComboFailed`, landed/full/floated-away); DOTween fly-in with one-frame layout delay; row grouping, idle-decay timeouts, locked rows, and wrong-input detection.
- Created `StreakVFXManager` (event-driven, no direct UI refs) and a custom HLSL `StreakGlow.shader` for UI panel glow in HDRP (Shader Graph not supported on Canvas Images). Installed DOTween via `.unitypackage`.

May 27

NEW

REVERTED

Streak visual juice

- Added a `UIPixelTrail` behind flying icons, icon land-punch overshoot, fail color-drain tint, and a combo-name pop wired through `OnComboCompleted(string)`.
- Prototyped a stacking depth illusion (completed rows shrink/fade behind new rows) and an idle pulse — both reverted to the stable state after the per-stack alpha couldn't be resolved cleanly.

May 29

NEW FIX

TMP dissolve shader for combo name

- Wrote a custom HLSL TMP SDF dissolve shader (`TMP/PSXDissolve, Dissolver.shader`) using procedural value noise and a baked posterized edge glow; driven by `TMPDissolveDriver` animating `_DissolveAmount` 0→1.
- Wired into `StreakUI` as a hold-then-dematerialize flow on combo complete. Dropped the “interrupt on next attack” path after stack traces showed `RaiseComboCompleted` and `RaiseAttackStartedWithIcon` fire in the same frame.

May 29

NEW FIX

Dodge attack combo integration

- Added dedicated dodge-attack combos (DH / DL). Fixed L-mashing step-skips with a `_pendingStepIndex` buffer that only advances `currentStepIndex` in `ConsumeBuffer`.
- Solved single-step dodge combos not retiring their HUD row by raising `RaiseComboCompleted` on the fresh-attack path in `OnAttackCommitted` when the opener is also the finisher — no new prefab needed.

Architecture at a glance

- **Event-driven Mediator** — all inter-system communication routes through `CombatEvents`; no direct script-to-script references. VFX, audio, UI and AI subscribe independently.
- **Data-driven combat** — attacks, combo sequences and weapon loadouts are `ScriptableObject`s; adding a weapon is asset creation, not code.
- **Coroutine-driven flow** — `AttackAnimationDriver` owns phase timing (startup → impact → recovery) and clip-relative hitbox windows.
- **Designer tooling** — custom Editor window with a visual timeline, draggable markers and phase-speed presets so non-programmers can tune feel.